

Offline Reinforcement Learning for Autonomous Driving with Real World Driving Data

Xing Fang^{1,3}, Qichao Zhang^{1,2,4}, Yinfeng Gao^{1,5}, Dongbin Zhao^{1,2,4}

Abstract—Since traditional reinforcement learning (RL) approaches need active online interaction with the environment, previous works are mainly investigated in the simulation environment rather than the real world environment, especially for safety-critical applications. Offline RL has recently emerged as a promising data-driven learning paradigm to learn a policy from offline dataset directly. It seems that offline RL is well suited for autonomous driving, as it is feasible to collect offline naturalized driving dataset. However, it remains unclear how to deploy offline RL with real world driving dataset only including observation data, and whether current offline RL algorithms work well to learn a driving policy than imitation learning? In this paper, we provide an offline RL benchmark for autonomous driving including the dataset, baselines, and a data driven simulator¹. First, we summarize and introduce the popular offline RL baseline methods. Then, we construct an offline RL dataset for the car following task based on the real world driving dataset INTERACTION. A data driven simulator is applied to obtain augmented data and test the driving policy. Further, we deploy four popular offline algorithms and analyze their performances under different datasets including real world driving data and augmented data. Finally, related conclusions and discussions are given to analyze the critical challenge for offline RL in autonomous driving.

I. INTRODUCTION

Reinforcement learning (RL) is a powerful semi-supervised learning framework, which has been used to address the decision-making problem successfully such as Go [1], nuclear fusion [2] and so on. Recently, RL has achieved widely attention in the decision-making [3] and control task [4] of autonomous driving (AD). However, since RL algorithms need to interact with the environment, previous works are mainly deployed in the simulation environment considering the safety and efficiency [5], [6], [7]. In fact, the sim2real gap, which is introduced by transferring the trained policy in the simulator to the real world environment, is one of key problems to hinder the application of online RL in real world.

*This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grants No. 62173325, and the Beijing Municipal Natural Science Foundation under Grants L191002.

¹The State Key Laboratory of Management and Control for Complex Systems, Institute of Automation, Chinese Academy of Sciences, Beijing 100190, China. Email: zhangqichao2014@ia.ac.cn

²School of Artificial Intelligence, University of Chinese Academy of Sciences, Beijing 100049, China.

³School of Mathematical Sciences, University of Electronic Science and Technology of China, Chengdu 611731, China.

⁴Peng Cheng Laboratory, Shenzhen, China.

⁵School of Automation and Electrical Engineering, University of Science and Technology Beijing, Beijing, 100083, China.

¹Code: <https://github.com/weiaif/OfflineRL-INTERACTION>

Offline RL, which utilizes only collected offline data without additional online interaction with the environment, offers the promise to address the safety-critical autonomous driving task [8], [9]. With recent advances, offline RL algorithms have made great progress, which are based on the acknowledged D4RL [10] and RL Unplugged [11] dataset. In particular, D4RL builds an offline dataset for lateral control of autonomous driving. Unfortunately, the data is all derived from the CARLA simulator [12] rather than the real world. The biggest problem with real world driving datasets such as Argoverse [13], INTERACTION [14] is that they only contain observation data and lack the action and reward data information required by offline RL. Recently, more and more researchers have pay attention to real world dataset of offline RL such as offline human demonstrations from robot manipulation and navigation [15], [16]. However, there are few works that have considered training the offline RL algorithms with real word dataset only including observation data for AD.

In this paper, we provide an offline RL benchmark for autonomous driving with real world driving data. To analyze the impact of data on the algorithm, a data driven driving simulator is applied to obtain augmented data and test the driving policy. Current popular offline RL algorithms are deployed based on our proposed offline dataset. The main contributions of this paper are summarized as follows.

- 1) Different from the synthetic driving dataset in [10], we provide a driving dataset from real world naturalistic driving data [14] for offline RL. In addition, a data-driven driving simulator is applied which can be used to generate augmented data for improving the performance of offline RL.
- 2) We present an offline RL benchmark for autonomous driving by deploy four popular offline learning algorithms on the provided datasets and simulator. By comparing their performances under three offline driving datasets, we analyze and discuss the critical challenge for offline RL in autonomous driving.

II. RELATED WORK

A. Offline reinforcement learning

Offline RL aims to learn a policy from a pre-collected and static dataset without online environment interactions [8], [9]. The major challenge in offline RL is the distribution shift. Since the algorithms are trained on the offline dataset generated from the behavior policy, the learned policy might execute out-of-distribution (OOD) actions [17], [18]. Traditional RL can recover from this distribution shift by online

exploration in the environment. For offline RL algorithms, the solutions of distribution shift problem are mainly divided into three categories including policy constraint, conservative estimation and uncertainty estimation. Policy constraint methods can be divided into explicit and implicit policy constraints, where explicit methods estimate the behavioral policy π_β and constrain the learned policy π_θ to stay close to π_β such as BCQ [17]. The implicit methods use a modified objective and rely on samples from π_β . For example, TD3+BC [19] adds a behavior cloning term to the policy updating of Twin Delayed Deep Deterministic Policy Gradient (TD3) [20]. Conservative estimation methods tune the learned function by adding a penalty term. As a representative method, CQL [21] learns the Q function that lower-bounds its values. In addition, uncertainty-based offline RL methods can relax the constraints on the learned policy in low-uncertainty regions by incorporating a reward penalty [22], [23], which are usually based on model-based RL.

B. Datasets for offline RL

Currently common offline datasets for offline RL are D4RL [10] and RL Unplugged [11], where those data are mainly from replay buffers and collected trajectories during online RL training process. In addition, NeoRL [24] is a near real world benchmark for offline reinforcement learning including industrial control, stock trading, energy storage control tasks. For the robot manipulation application, a real world dataset from human demonstrations is given to train offline RL [15]. Recently, Yarats et al. [25] consider the collection of informative data for offline RL with unsupervised RL, which encourages researchers to explore the importance of data for offline RL. Unfortunately, there is no an offline RL benchmark for autonomous driving with real world driving data. Motivated by this, we use the naturalized driving trajectory dataset INTERACTION [14] to construct an offline RL benchmark for autonomous driving including the dataset for training and the simulator for testing.

III. THE OFFLINE RL BENCHMARK FOR AUTONOMOUS DRIVING

A. Reinforcement learning

In reinforcement learning, the process that an agent interacts with the environment is typically described as a Markov Decision Process (MDP) $\langle S, A, P, R, \gamma \rangle$, where S represents the set of states, A represents the set of actions, $P(\cdot|s, a) : S \times A \times S \rightarrow [0, 1]$ represents the dynamic transition model, $R(s, a)$ represents the reward function, and $\gamma \in [0, 1]$ is the discount factor. The goal of RL is to learn a policy $\pi : S \rightarrow A$ that maximizes the expectation of the sum of discounted rewards, known as the return $G_t = \sum_{i=t}^{\infty} \gamma^{i-t} R_i$. Each policy π has a corresponding state-action value function (also known as Q function), which denotes the expected return $Q(s, a)$ when following the policy π after taking an action a in state s .

$$Q(s, a) = \mathbb{E}[G_t | s_t = s, a_t = a] \quad (1)$$

For traditional Q-learning, a behavior policy such as ϵ -greedy policy is used to interact with the environment. Based on off-policy evaluation, the optimal Q function is usually be approached with a neural network approximator Q_θ based on the following equation:

$$Q_\theta(s, a) \leftarrow \mathbb{E} \left(R(s, a) + \gamma \max_{a' \in A} Q_\theta(s', a') \right) \quad (2)$$

Then, the optimal policy can be obtained by

$$\pi^*(s) = \arg \max_{a \in A} Q_\theta(s, a) \quad (3)$$

For large or continuous state and action spaces, the experience replay [26] and target network techniques proposed in DQN [27] are used to improve the stability of training.

$$Q_\theta(s, a) \leftarrow \mathbb{E} \left(R(s, a) + \gamma \max_{a' \in A} Q_{\theta'}(s', a') \right) \quad (4)$$

where the tuple (s, a, R, s') is sampled from an experience replay buffer, and θ' is the parameter of target network which is updated to the current Q network parameter θ after a fixed number of time steps.

B. Popular offline RL algorithms

Offline RL aims to learn a policy from offline dataset **D**. In the following, several popular offline RL methods are introduced.

1) *batch-constrained deep Q-learning (BCQ)*: BCQ is a representative work of explicit policy constraint, which first learns a state-conditioned variational auto-encoder (VAE) model $G_\omega(s)$ to mimic the behavior policy of the offline dataset **D**. Then, a perturbation network $\xi_\phi(s, a, \Phi)$ is proposed to output an adjustment to the action a in the perturbation range $[-\Phi, \Phi]$, which can access to actions in a constrained region. Further, BCQ samples mini-batch tuples (s, a, R, s') from **D**, and obtains sampled actions $\hat{A} = \{a_i + \xi_\phi(s', a_i, \Phi) | a_i \sim G_\omega(s')\}_{i=1}^N$ with the VAE and perturbation models. To penalize uncertainty over future states, the modified Clipped Double Q-learning is used to estimate the Q value with two Q networks $Q_{\theta_1}, Q_{\theta_2}$. The target Q value Q_{target} is estimated with two target Q networks $Q_{\theta'_1}, Q_{\theta'_2}$ using

$$Q_{target} = R + \gamma \max_{a_i \in \hat{A}} \left[\lambda \min_{j=1,2} Q_{\theta'_j}(s', a_i) + (1 - \lambda) \max_{j=1,2} Q_{\theta'_j}(s', a_i) \right] \quad (5)$$

where λ is a tradeoff factor. Finally, the Q network is trained by minimizing the Bellman error loss $\arg \min_\theta \sum (Q_{target} - Q_\theta(s, a))^2$. With the converged Q network, the policy can be obtained by

$$\pi(s) = \arg \max_{a_i + \xi_\phi(s, a_i, \Phi)} Q_\theta(s, a_i + \xi_\phi(s, a_i, \Phi)), \{a_i \sim G_\omega(s)\}_{i=1}^N \quad (6)$$

2) *conservative Q-learning (CQL)*: Different from the explicit policy constraint in BCQ, CQL places the penalty on the Q-function. It aims to learn a conservative Q-function such that the expected value of a policy under this Q-function lower-bounds its true value. First, to prevent overestimation

of the value under current policy $\pi_\theta(\cdot)$, an additional Q value term $\mathbb{E}_{s \sim \mathbf{D}, a \sim \pi_\theta(a|s)} Q(s, a)$ is minimized alongside a standard Bellman error objective. Then, another additional Q-value maximization term $\mathbb{E}_{s \sim \mathbf{D}, a \sim \hat{\pi}_\beta(a|s)} Q(s, a)$ under the data generated by $\pi_\beta(a|s)$ is introduced to tighten the value lower-bound, where $\hat{\pi}_\beta(a|s) = \frac{\sum_{s, a \in \mathbf{D}} \mathbf{1}[s=s, a=a]}{\sum_{s \in \mathbf{D}} \mathbf{1}[s=s]}$ denotes the empirical behavior policy at all states $s \in \mathbf{D}$. Specifically, the iterative update for training the Q-function can be described as:

$$Q(s, a) \leftarrow \arg \min_Q \frac{1}{2} \mathbb{E}_{s, a, s' \sim \mathbf{D}} (Q(s, a) - Q_{target})^2 + \lambda \left(\mathbb{E}_{s \sim \mathbf{D}, a \sim \pi_\theta(a|s)} Q(s, a) - \mathbb{E}_{s \sim \mathbf{D}, a \sim \hat{\pi}_\beta(a|s)} Q(s, a) \right) \quad (7)$$

Note that Q_{target} in CQL can be obtained based on top of existing deep Q-learning and actor-critic implementations.

3) *TD3 with behavioral cloning (TD3+BC)*: TD3+BC is a representative offline RL algorithm for implicit policy constraint with simply adding a behavior cloning term to the policy update of an online RL algorithm (TD3):

$$\pi = \arg \max_{\pi} \mathbb{E}_{(s, a) \sim \mathbf{D}} [\lambda Q(s, \pi(s)) - (\pi(s) - a)^2] \quad (8)$$

where $\lambda = \frac{\alpha}{\frac{1}{N} \sum_{(s_i, a_i)} |Q(s_i, a_i)|}$ is a normalization term based on the average absolute value of Q to control the balance between RL (in maximizing Q) and imitation (in minimizing the BC term).

C. Construct offline dataset from real world driving data

To construct an offline RL dataset for AD, we use the real world driving dataset INTERACTION [14] which is widely used for the trajectory prediction task. INTERACTION dataset contains real driving trajectories with the time, shape and motion information of the vehicle from different highly interactive and complex urban scenarios. In this paper, we consider the car following task simplified to longitudinal control in the DR_CHN_Merging_ZS (ZS) map as shown in Fig. 1. In the following, we give the processing to obtain the corresponding MDP tuple (s, a, R, s') for offline dataset.

State: We use a 49-dimension vector to represent the current state of the ego vehicle with the form $s = [s_e, s_{mask}, s_1, s_2, \dots, s_5]$. The state vector s can be divided into three major parts as displayed in TABLE I: s_e contains the feature of the ego vehicle including its shape (length and width), current speed value, previous speed value, current coordinate and the predict coordinate in next moment if the ego vehicle maintains current heading and speed. s_{mask} represents the existence of ego vehicle and 5 social vehicles to simplify the calculation. $s_1, \dots, s_5$ describe the feature of 5 social vehicles including the shape, current speed value, coordinate relative to ego vehicle, and the cosine and sine value of social vehicle's current heading relative to the ego vehicle. The next state s' can be obtained in the same way.

In addition, we use a filter to choose social vehicles, which is summarized into the following three rules:

- vehicle whose distance to ego vehicle is less than 30m.
- vehicle which is not behind ego vehicle more than 12m.
- the closest 5 vehicles are considered as social vehicles.

TABLE I
STATE REPRESENTATIONS

Component	Description	Length
s_e	ego's shape and motion information	1*8
s_{mask}	mask	1*6
$s_i (i = 1, 2, \dots, 5)$	social vehicles' shape and motion	5*7
s	complete state vector	49

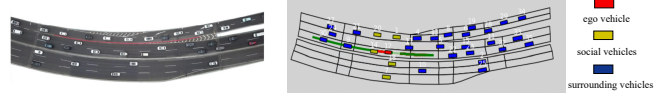


Fig. 1. The highway scenarios with ZS map in INTERACTION

Action: For the car following task, the control action can be desired longitudinal velocity of the ego vehicle. Since the trajectories from INTERACTION are collected from human drivers, we can consider the ego vehicle's longitudinal speed in the next timestep as its action a . Note that the lateral behavior of the ego vehicle follows the original trajectory in the dataset.

Reward: The design of the reward should satisfy demands of the car following task, which can be summarized as safety and efficiency. To improve the safety of ego vehicle, we use a collision penalty term, i.e., $r_c = 0$ (if no collision) or -100 (if collision happens). In order to improve the efficiency, it is expected that the speed of ego vehicle should be fast. Therefore, we use a speed reward $r_v = 0.5 * v_t / v_{max}$, where v_t is the velocity of the ego vehicle at timestep t , and $v_{max} = 25m/s$ which is statistically obtained from the dataset. Thus, the total reward is $R = r_c + r_v$.

By combining the above data, we can achieve the tuple (s, a, R, s') for offline RL training. Further, the offline RL data can be obtained by processing all the trajectories in the INTERACTION, which is called as expert offline data.

D. Simulator

To test offline RL algorithms, a driving simulator that can replay the scenarios from INTERACTION and control the ego vehicle is required. In this paper, we apply our previous developed data-driven simulator with three-layer architecture based on the INTERACTION dataset. The brief description about the three-layer architecture is given as follows.

- To achieve dynamic scenarios replay, the bottom layer is designed to load centimeter-accurate HD Map files for the display of map and recorded_trackfiles for vehicles from the INTERACTION dataset. In this layer, we also provide the agent's kinematic bicycle model [28] and geometry calculation functions.
- To control the ego vehicle for testing, we first need to obtain the observation state at each timestep from the controlled vehicle perspective. The middle layer is responsible for provide the observation state s described in TABLE I. Then, based on the server-client pattern in the top layer, the offline RL on the client can obtain the observation state, and output an action. Finally, this

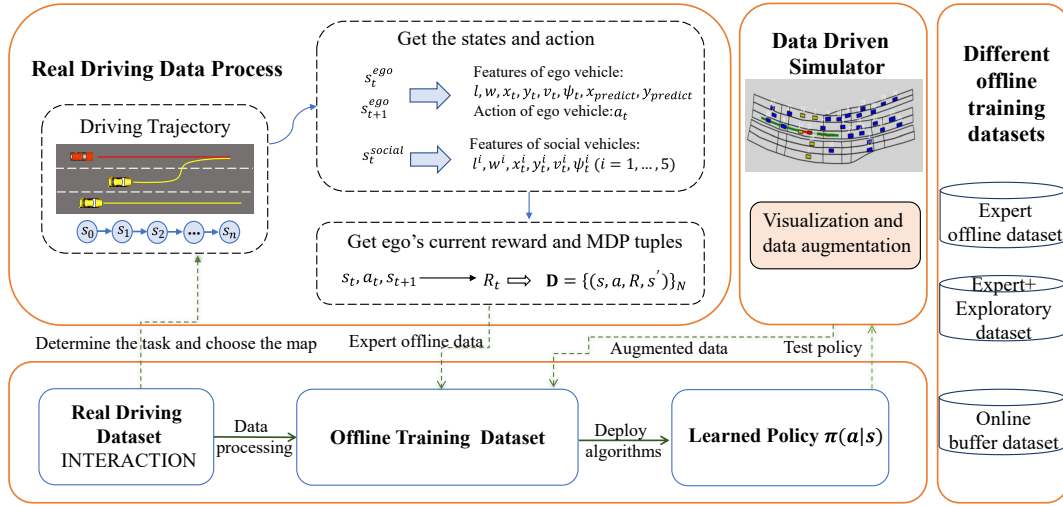


Fig. 2. The offline RL framework for autonomous driving based on INTERACTION dataset

action is transmitted to the controlled ego vehicle on the server, the next observation state can be obtained with the kinematic bicycle model.

Since there is no collision data for the expert offline dataset, we can use this simulator to collect augmented data with failed trajectories and analyze the impact of data on the offline RL algorithms. The offline RL framework for autonomous driving is illustrated in Fig. 2.

IV. EXPERIMENTS

A. Offline training dataset

1) *expert offline dataset*: We process 100 real world driving trajectories in the ZS highway map from INTERACTION to create corresponding expert offline data. First, we get $s_e(t)$ state and action a_t of the ego vehicle through its driving trajectory. Then s_{mask} and $s_i(t) (i = 1, 2, \dots, 5)$ state of social vehicles can be obtained by trajectories of social vehicles, which have the same timestamp with the ego vehicle and satisfy the above filter. Finally, we obtain 14346 MDP tuples (s, a, R, s') as the expert offline data.

2) *expert+exploratory dataset*: We generate exploratory data through our developed simulator. First, the scenario including the map and vehicles information is initialized in the simulator based on the original dataset. Then we select one vehicle arbitrarily as the controlled ego vehicle, and deploy the random policy to control the longitudinal velocity. All the other vehicles simply follow the original trajectories from the dataset. In this way, there will be a lot of synthetic collision data. We mix expert offline data and exploratory data to get expert+exploratory dataset with $1e5$ MDP tuples.

3) *online buffer dataset*: We also train an online Deep Deterministic Policy Gradient (DDPG) [29] agent by adding $N(0, 0.1)$ Gaussian noise to actions for $1e5$ time steps in the simulator, and store all experienced transitions. In this way, we also obtain $1e5$ MDP tuples including various states and actions.

B. Baselines and implementation details

1) *baselines*: In the experiments, we choose the popular offline RL algorithms BCQ [17], CQL [21], TD3+BC [19], and the typical imitation learning method BC [30] as baselines. Note that BC is trained to learn the mapping function $\pi_\theta(s)$ from the state to action directly based on the offline dataset via the objective $\arg \min_\theta \mathbb{E}_{(s,a) \sim \mathbf{D}} \|\pi_\theta(s) - a\|^2$.

Here, we list corresponding hyperparameters of our baselines. For BCQ, we use $N = 10$ sampled actions each training and set perturbation range to 0.05, tradeoff λ to 0.75, learning rate to $1e-3$; for BC we use the framework of VAE and learning rate is $1e-3$. For CQL, λ is set to 0.5 and learning rate is $5e-4$. For TD3+BC, the hyperparameters α is set to 2.5, N is the size of mini-batch and learning rate is $3e-4$. The batch size of all algorithms is 128 and we run the experiments on GTX 1080 Ti $\times 4$ GPU.

2) *implementation details*: To compare the performances of baselines, each offline agent is trained for 30 epochs, where each epoch consists of 5000 gradient steps. Then we evaluate the policy after each epoch by running 100 rollouts in the simulator. Each timestep in the simulator lasts 0.1s, and the ego vehicle is controlled until a collision occurs, the end of the real trajectory is reached, or the real trajectory log time reaches.

C. Performance comparison

We conduct experiments on three offline training datasets with different characteristics for all baselines. The average return of baselines after each epoch is shown in Fig. 3. And the average episode return and success rate (collision-free rate) in the last epoch are reported in Table II.

From Fig. 3(a), we can see that offline RL algorithms have poor performances and are even not capable to BC algorithm on the expert offline dataset. In fact, this phenomenon also exists in robot manipulation with offline human demonstrations [15]. Then we obtain the first empirical insight: **BC is preferred than offline RL with only expert data for autonomous driving task in dense traffic.**

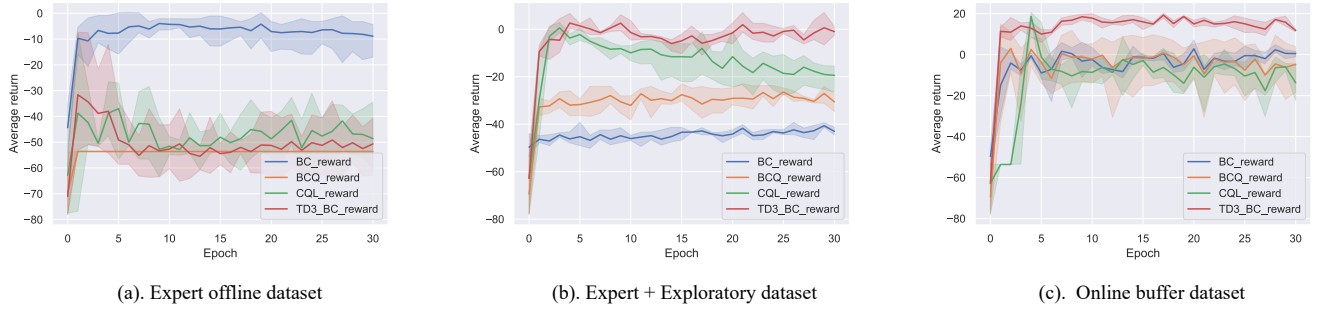


Fig. 3. The average return of baselines on three offline training datasets.

From Fig. 3(b), we find that the performances of offline RL algorithms are improved significantly with the augmented exploratory data, while the performance of BC degrades due to the introduction of collision data. The performance of TD3+BC is better than CQL and BCQ on the mixed dataset including expert and exploratory data, and is close with BC on the expert offline dataset in Fig. 3(a). Then we obtain the second empirical insight: **Offline RL is a better choice for the mixed dataset with better coverage in the state-action space.**

In order to further investigate the impact of dataset for offline RL, we also train our baselines on the online buffer dataset. According to Fig. 3(c) and Table II, it can be seen that the performances of offline RL algorithms on the online buffer dataset are highly improved. Then we obtain the third empirical insight: **The data distribution of online training buffer is more suitable for offline RL to learn an optimal policy under the designed reward function.**

TABLE II
RESULTS TRAINED WITH DIFFERENT DATASET

Algorithm	Dataset	Average Return	Success Rate
BC[30]	Expert offline dataset	-8.9	0.73
	Expert+Exploratory	-43.0	0.42
	Online buffer dataset	0.5	0.81
BCQ[17]	Expert offline dataset	-53.6	0.33
	Expert+Exploratory	-30.6	0.55
	Online buffer dataset	-4.8	0.76
CQL[21]	Expert offline dataset	-48.6	0.38
	Expert+Exploratory	-19.4	0.63
	Online buffer dataset	-13.8	0.68
TD3+BC[19]	Expert offline dataset	-50.7	0.35
	Expert+Exploratory	-1.0	0.80
	Online buffer dataset	11.7	0.91

D. Analysis and discussion

These above empirical insights inspire us to study the characteristic of learning from demonstration, data augmentation and the state-action distribution of different datasets.

1) *learning from demonstration*: While recent offline RL algorithms have shown excellent performances based on generated datasets from simulation [17], [21], we empirically find that they fail to learn well from human demonstration data for AD. This conclusion is also consistent with [15] for robotic manipulation task. According to the practical

insight 2 in [31], offline RL is preferred over BC even with expert or near-expert data when the environment has a few critical states. We think the possible reason for our first empirical insight is that the critical states (near-to-collision states) happen frequently in dense traffic, where precisely taking the expert action is necessary.

2) *data augmentation*: The above analysis indicates that the key to solving AD tasks is making a right decision in the critical state, i.e., avoid collision. However, there is no critical-collision tuples in the expert offline dataset, which results offline RL agents fail to learn the consequence of collision. Thus we provides failed trajectories by exploratory agent, which enable the offline RL agent to learn what not to do. Here we transform the 49-dimensional state into two-dimensional vector using t-SNE [32]. And as shown in Fig. 4, exploratory data indeed covers the missing state space of expert offline data, which gives an explanation for the second empirical insight.

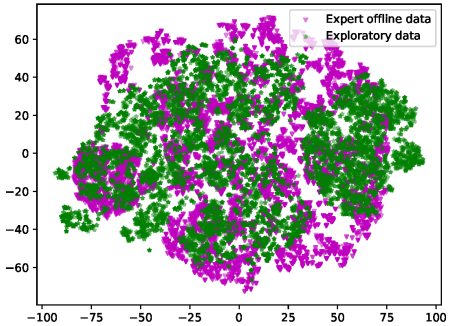


Fig. 4. State space visualization of expert offline data and exploratory data.

3) *state-action distribution of different datasets*: To show the difference of different datasets, the state-action density is given in Fig. 5. Note that we use t-SNE to transform the 49-dimensional state into one dimensional vector. From Fig. 5(a) we can see that drivers in the real world drive naturally, whose actions are mainly in the average range satisfying their intrinsic motivation. As for online buffer dataset in Fig. 5(b), the state-action distribution is highly correlated to the designed reward function. Specifically, online trained agent performs aggressive with high speed to obtain the speed reward r_v and also brakes in time to prevent collision. The reason is that the online buffer data is generated by the online DDPG policy to maximize the cumulative reward. In other words, the online buffer dataset

is a noisy-optimal dataset while the expert offline data is a suboptimal dataset under this designed reward function. This gives an explanation for the third empirical insight.

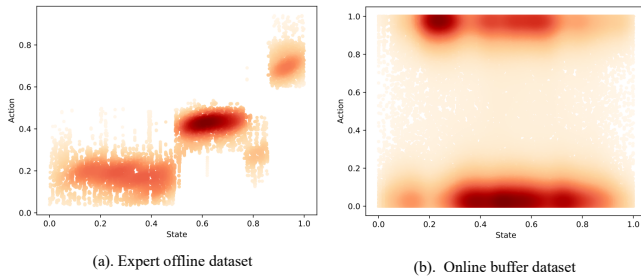


Fig. 5. State-action density of different datasets. Darker colors represent more frequent state-action pairs.

V. CONCLUSIONS

In this study, we propose an offline RL benchmark for autonomous driving with real world driving data and augmented data. The popular offline RL algorithms are deployed and compared in our constructed offline RL dataset and data-driven simulator. The results show that directly deploying offline RL algorithms with real world driving data, lacking collision data, yields frustrating results. We also investigate the effect of data augmentation and find that the performances of offline RL algorithms are improved with augmented exploratory data. In addition, we reveal that offline RL achieves excellent performance on online training buffer because of the match of data distribution and the designed reward function. Finally, we would like to share possible directions for offline RL in autonomous driving. In the future, we will consider scenarios with larger amounts of data, deploy advanced algorithms with uncertainty estimation technique and further investigate the performance of offline RL with the reward function generated by offline expert trajectories and inverse reinforcement learning.

REFERENCES

- [1] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, and D. Hassabis, "Mastering the game of Go without human knowledge," *Nature*, vol. 550, no. 7676, pp. 354–359, 2017.
- [2] J. Degraeve, F. Felici, J. Buchli *et al.*, "Magnetic control of tokamak plasmas through deep reinforcement learning," *Nature*, vol. 602, no. 7897, pp. 414–419, 2022.
- [3] J. Wang, Q. Zhang, and D. Zhao, "Highway lane change decision-making via attention-based deep reinforcement learning," *IEEE/CAA Journal of Automatica Sinica*, vol. 9, no. 3, pp. 567–569, 2022.
- [4] D. Li, D. Zhao, Q. Zhang, and Y. Chen, "Reinforcement learning and deep learning based lateral control for autonomous driving," *IEEE Computational Intelligence Magazine*, vol. 14, no. 2, pp. 83–98, 2019.
- [5] P. R. Wurman, S. Barrett, K. Kawamoto *et al.*, "Outracing champion gran turismo drivers with deep reinforcement learning," *Nature*, vol. 602, no. 7896, pp. 223–228, 2022.
- [6] J. Wang, Q. Zhang, D. Zhao, and Y. Chen, "Lane change decision-making through deep reinforcement learning with rule-based constraints," in *2019 International Joint Conference on Neural Networks (IJCNN)*, 2019, pp. 1–6.
- [7] D. Li, D. Zhao, and Q. Zhang, "Reinforcement learning based lane change decision-making with imaginary sampling," in *2019 IEEE Symposium Series on Computational Intelligence*, 2019, pp. 16–21.
- [8] S. Levine, A. Kumar, G. Tucker, and J. Fu, "Offline reinforcement learning: Tutorial, review, and perspectives on open problems," *arXiv preprint arXiv:2005.01643*, 2020.
- [9] R. F. Prudencio, M. R. Maximo, and E. L. Colombari, "A survey on offline reinforcement learning: Taxonomy, review, and open problems," *arXiv preprint arXiv:2203.01387*, 2022.
- [10] J. Fu, A. Kumar, O. Nachum, G. Tucker, and S. Levine, "D4RL: Datasets for deep data-driven reinforcement learning," *arXiv preprint arXiv:2004.07219*, 2020.
- [11] C. Gulcehre, Z. Wang, A. Novikov *et al.*, "RL Unplugged: A suite of benchmarks for offline reinforcement learning," *arXiv preprint arXiv:2006.13888*, 2020.
- [12] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in *Conference on Robot Learning*. PMLR, 2017, pp. 1–16.
- [13] M. Chang, J. Lambert, P. Sangkloy *et al.*, "Argoverse: 3d tracking and forecasting with rich maps," in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 8740–8749.
- [14] W. Zhan, L. Sun, D. Wang *et al.*, "INTERACTION dataset: An International, Adversarial and Cooperative motion Dataset in Interactive Driving Scenarios with Semantic Maps," *arXiv preprint arXiv:1910.03088*, 2019.
- [15] A. Mandlekar, D. Xu, J. Wong *et al.*, "What matters in learning from offline human demonstrations for robot manipulation," in *5th Annual Conference on Robot Learning*, 2021.
- [16] H. Li, Q. Zhang, and D. Zhao, "Deep reinforcement learning-based automatic exploration for navigation in unknown environment," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 31, no. 6, pp. 2064–2076, 2020.
- [17] S. Fujimoto, D. Meger, and D. Precup, "Off-policy deep reinforcement learning without exploration," in *International Conference on Machine Learning*, 2019, pp. 2052–2062.
- [18] A. Kumar, J. Fu, M. Soh, G. Tucker, and S. Levine, "Stabilizing off-policy q-learning via bootstrapping error reduction," in *Advances in Neural Information Processing Systems*, 2019, pp. 11 784–11 794.
- [19] S. Fujimoto and S. S. Gu, "A minimalist approach to offline reinforcement learning," in *Advances in Neural Information Processing Systems*, 2021.
- [20] S. Fujimoto, H. Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *International Conference on Machine Learning*, 2018, pp. 1582–1591.
- [21] A. Kumar, A. Zhou, G. Tucker, and S. Levine, "Conservative q-learning for offline reinforcement learning," in *Advances in Neural Information Processing Systems*, 2020, pp. 1179–1191.
- [22] R. Kidambi, A. Rajeswaran, P. Netrapalli, and T. Joachims, "MOREL: Model-based offline reinforcement learning," in *Advances in neural information processing systems*, 2020, pp. 21 810–21 823.
- [23] T. Yu, G. Thomas, L. Yu, S. Ermon, J. Y. Zou, S. Levine, C. Finn, and T. Ma, "MOPO: Model-based offline policy optimization," in *Advances in Neural Information Processing Systems*, 2020, pp. 14 129–14 142.
- [24] R. Qin, S. Gao, X. Zhang *et al.*, "NeoRL: A near real-world benchmark for offline reinforcement learning," *arXiv preprint arXiv:2102.00714*, 2021.
- [25] D. Yarats, D. Brandfonbrener, H. Liu, M. Laskin, P. Abbeel, A. Lazaric, and L. Pinto, "Don't change the algorithm, change the data: Exploratory data for offline reinforcement learning," *arXiv preprint arXiv:2201.13425*, 2022.
- [26] B. Wang, D. Zhao, and J. Cheng, "Adaptive cruise control via adaptive dynamic programming with experience replay," *Soft Computing*, vol. 23, no. 12, pp. 4131–4144, 2019.
- [27] V. Mnih, K. Kavukcuoglu, D. Silver *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [28] P. Polack, F. Althé, B. d'Andréa Novel, and A. de La Fortelle, "The kinematic bicycle model: A consistent model for planning feasible trajectories for autonomous vehicles?" in *2017 IEEE Intelligent Vehicles Symposium (IV)*. IEEE, 2017, pp. 812–818.
- [29] T. P. Lillicrap, J. J. Hunt, A. Pritzel *et al.*, "Continuous control with deep reinforcement learning," in *International Conference on Learning Representations*, 2016.
- [30] D. Pomerleau, "Alvin : An autonomous land vehicle in a neural network," in *Advances in Neural Information Processing Systems*, 1989, pp. 305–313.
- [31] A. Kumar, J. Hong, A. Singh, and S. Levine, "Should I run offline reinforcement learning or behavioral cloning?" in *International Conference on Learning Representations*, 2022.
- [32] L. Van der Maaten and G. Hinton, "Visualizing data using t-sne," *Journal of Machine Learning Research*, vol. 9, no. 11, 2008.